

تمرینات کلاسی هفته ششم - سری دوم

بخش 1 - یک نوت بوک/اسکرپت تازه باز کنید. این ایمپورت ها را اجرا کنید:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
np.random.seed(42)
```

1.1 - دو توزیع زیر را تعریف کنید:

```
girl_dist = stats.norm(4.0, 1.2)
boy_dist = stats.norm(7.0, 1.5)
```

هر دو تابع چگالی احتمال را روی

```
x = np.linspace(0, 12, 1000)
```

رسم کنید.

1.2 - $n = 5000$ نوزاد بساز (۷۰٪ دختر و ۳۰٪ پسر، آرایه‌ی X (ستونی) و y ، عدد ۰ یعنی دختر، ۱ یعنی پسر). داده را به آموزش/آزمون تقسیم کنید (۷۰/۳۰)، و یادتان باشد $\text{stratify}=y$ را بگذارید. یک $\text{KNeighborsClassifier}$ با $n_neighbors=15$ آموزش دهید. دقت روی داده آزمون را چاپ کنید.

1.3 - کد زیر خطا می دهد. دو باگ را پیدا کرده و درستش کنید.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
X = girl_dist.rvs(3500) # آرایه یک بعدی از سطوح هورمونی
y = [0]*3500
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)
knn = KNeighborsClassifier(n_neighbors=15)
knn.fit(X_train, y_train)
```

(راهنمایی: دو ایراد جدا از هم اینجا هست – یکی درباره شکلی که scikit-learn انتظار دارد، و یکی درباره این که آیا اصلاً این مجموعه داده چیز مفیدی برای یاد دادن به یک طبقه بند دارد یا نه.)

=====

بخش 2 – برازش یک خط

2.1 – دکتر لیند، دانه‌های قهوه را برشته می‌کند و می‌خواهد درصد کاهش وزن حین برشته شدن را از روی زمان برشته شدن (دقیقه) پیش‌بینی کند. داده مصنوعی زیر را بسازید:

```
import numpy as np
np.random.seed(0)
n = 100
roast_time = np.random.uniform(8, 18, n)
weight_loss = 2.0 + 0.9 * roast_time + np.random.normal(0, 1.5, n)
```

نمودار پراکندگی roast_time در مقابل weight_loss را رسم کنید.
یک مدل LinearRegression برازش دهید (یادتان باشد: roast_time باید به شکل ستونی درآید).
عرض از مبدا و شیب یادگرفته شده، مقدار MSE، و مقدار R^2 را چاپ کنید.

2.2_2.1 را دوباره انجام دهید، اما این بار نویز را به

```
np.random.normal(0, 4.0, n)
```

تغییر دهید (داده پرنویزتر). R^2 نسبت به 2.1 چطور تغییر می‌کند؟

=====

بخش 3

3.1 – برای هر سناریوی زیر، تصمیم بگیرید کدام ابزار بهترین گزینه است: رگرسیون خطی، رگرسیون لجستیک، KNN

- الف) پیش بینی قیمت فروش یک خانه از روی مترژ و تعداد اتاق خواب.
- ب) پیش بینی این که آیا یک تراکنش ورودی تقلبی است یا نه، در حالی که یک تخمین احتمالی هم می خواهیم.
- ج) طبقه بندی یک میوه جدید به عنوان رسیده یا نارس، با مقایسه رنگ و سفتی اش با میوه های مشابه از قبل برچسب خورده.
- د) تخمین فروش بستنی ماه آینده از روی میانگین دمای روزانه.

3.2 – یک مدل با واریانس بالا با داده‌ی آموزشی چیکار می‌کنه، و روی داده‌ی جدید چطور عمل می‌کنه؟
یک مدل با بایاس بالا چه مشکلی داره، و روی داده‌ی آموزشی و جدید چطور عمل می‌کنه؟

3.3 – درست یا غلط

- الف) «KNN در زمان آموزش، یک معادله ریاضی ثابت یاد می‌گیرد، شبیه رگرسیون خطی.»
- ب) « R^2 برابر با صفر همیشه یعنی مدل کاملاً بی‌فایده است و هیچ اطلاعاتی نمی‌دهد.»

=====

بخش 4 – رگرسیون خطی

4.1_ داده مصنوعی با رابطه‌ی

$$y = 7x - 10 + \text{نویز}$$

بسازید

```
np.random.normal(0, 8, n), n=80
```

بازه‌ی x بین ۰ تا ۲۰. یک LinearRegression فیت کنید و ببینید w و b یادگرفته‌شده چقدر به ۷ و ۱۰- نزدیکن.

4.2_ از diabetes_prediction_dataset1.csv فقط دو ستون age و blood_glucose_level رو به‌عنوان X ، و ستون bmi رو به‌عنوان y در نظر بگیرید (صرفاً برای تمرین رگرسیون، نه هدف اصلی دیتاست). یک LinearRegression فیت کنید و R^2 رو چاپ کنید.

=====

بخش 5 – سیگموید و سافت مکس

5.1_ تابع sigmoid رو برای

```
z = [-10, -3, 0, 3, 10]
```

تست کنید. این بار به جای آستانه‌ی معمول ۰.۵، از آستانه‌ی ۰.۳ استفاده کنید – یعنی اگر $\text{sigmoid}(z) > 0.3$ بود کلاس ۱ پیش‌بینی بشه. برای هر z بگید با این آستانه‌ی جدید چه کلاسی پیش‌بینی می‌شه.

5.2_ تابع softmax رو برای امتیازهای خام $[0.2, 0.2, 0.2, 0.2]$ (۴ کلاس، همه برابر) تست کنید. خروجی چی می‌شه؟ چرا منطقیه؟

=====

بخش 6 – رگرسیون لجستیک روی داده‌های واقعی دیابت

6.1_ از `diabetes_prediction_dataset1.csv` فقط سه ستون عددی `bmi`، `HbA1c_level` و `blood_glucose_level` رو به عنوان ویژگی انتخاب کنید (بدون `gender` و `smoking_history`). یک `LogisticRegression` فیت کنید و دقتش رو چاپ کنید.

راهنمایی: قبل از فیت کردن، حتماً داده رو با `stratify=y` به آموزش/آزمون تقسیم کنید، بعد با `StandardScaler` مقیاس‌بندی کنید (چون رگرسیون لجستیک از گرادیان دیسنت استفاده می‌کنه و به مقیاس ویژگی‌ها حساسه. در سوالات بعدی ما از این داده‌ها قراره استفاده کنیم، بنابراین اسامی این داده‌های مقیاس شده را `X_tr_s`، و دقت رو با `accuracy_score` روی داده‌ی آزمون (نه آموزش) حساب کنید.

6.2_ از `diabetes_dataset2.csv` فقط چهار ستون `HighBP`، `BMI`، `GenHlth`، `Age` رو انتخاب کنید. یک `LogisticRegression` فیت کنید و به جای فقط `accuracy`، از `classification_report` استفاده کنید تا `precision` و `recall` هر کلاس رو ببینید.

6.3_ روی مدل سؤال 6.1، ماتریس درهم‌ریختگی رو چاپ کنید و دستی حساب کنید accuracy، precision و recall از روی همون چهار عدد (بدون استفاده از تابع آماده‌ی sklearn برای این سه معیار، یعنی از ravel()) استفاده کنید و فرمولهای خود accuracy, precision , recall را بنویسید).

=====

بخش 7 – تابع هزینه‌ی کراس-انترופی

7.1_ این بار برای $y=0$ (بیمار واقعاً دیابت نداره)، loss رو برای دو پیش‌بینی مقایسه کنید: $p=0.1$ (مدل مطمئنه که دیابت نداره) و $p=0.8$ (مدل اشتباه و مطمئنه که داره). کدوم loss بزرگ‌تره؟ (راهنمایی: تابعی به اسم

```
cross_entropy_loss(p, y)
```

را تعریف کنید و فرمول کراس انترופی را داخلش بنویسید).

=====

بخش 8 – درخت تصمیم

8.1_ روی diabetes_prediction_dataset1.csv (همون داده‌ی آموزش/آزمون مقیاس‌بندی‌شده‌ای که در ۶.۱ ساختید (X_{tr}, y_{tr})), دو تا درخت فیت کنید: یکی با `criterion='gini'` و یکی با `criterion='entropy'` (هر دو با `max_depth=5`). دقت هر دو روی داده‌ی آزمون مقایسه کنید با `accuracy_score`

8.2_ یک دیتای غیرخطی جدید بسازید (نه XOR): «دایره‌ای» – یعنی برجسب ۱ باشه اگه

$$x_1^2 + x_2^2 < 4$$

وگرنه ۰

(با X تصادفی بین -۳ و ۳، ۲۰۰ نمونه). یک

```
DecisionTreeClassifier(max_depth=4)
```

فیت کنید و مرز تصمیمش رو با `DecisionBoundaryDisplay` رسم کنید.

(راهنمایی: برای ساخت دیتای دایره‌ای، اول X_c رو با

```
np.random.uniform(-3, 3, (n, 2))
```

بسازید (یعنی یک آرایه‌ی $n \times 2$ که هر ستونش بین -3 و 3 ه)، بعد برچسب y_c رو با یه شرط منطقی مستقیم روی دو ستون X_c بسازید

```
(Xc[:,0]**2 + Xc[:,1]**2 < 4)
```

و با

```
.astype(int)
```

به ۱/۰ تبدیلش کنید. برای رسم مرز، از

```
DecisionBoundaryDisplay.from_estimator(model, X, ...)
```

استفاده کنید – دقیقاً همون تابعی که تو نوت‌بوک کلاس برای رسم مرز XOR و moons استفاده شده بود. (

8.3_ روی همون y_c , X_c سؤال ۸.۲ (بدون train/test split، چون هدف فقط مقایسه‌ی شکل مرز نه ارزیابی واقعی)، یک LogisticRegression هم فیت کنید و دقتش رو با دقت درخت تصمیم مقایسه کنید. چرا انتظار دارید یکی خیلی بهتر از اون یکی عمل کنه؟ (از accuracy_score استفاده کنید) در واقع چون مرز واقعی یک دایره‌ست (غیرخطی)، رگرسیون لجستیک که فقط می‌تونه یک خط/ابریصفحه بکشه، عملکرد؟..... داره؛ درخت تصمیم چون می‌تونه با چندین شرط تو-در-تو ناحیه رو محاصره کنه، خیلی؟..... عمل می‌کنه.

8.4_ با یه حلقه، برای min_samples_leaf برابر ۱، ۵، ۲۰، و ۵۰، یک درخت روی

```
make_moons(n_samples=300, noise=0.25, random_state=7)
```

فیت کنید (بدون محدودیت عمق) و دقت روی همون داده رو چاپ کنید.

8.5_ برای max_depth از ۱ تا ۱۰، یک درخت روی Diabetes_prediction_dataset1.csv همون داده‌ی آموزش/آزمون مقیاس‌بندی‌شده‌ای که در ۶.۱ ساختید (X_{tr_s} , y_{tr}) فیت کنید و دقت روی داده‌ی **آزمون** (نه آموزش) رو برای هرکدام ثبت و رسم کنید. کدوم عمق بهترین دقت آزمون رو می‌ده؟

=====

بخش 9 – خوشه‌بندی

9.1_ با Make_blobs

(۴۰۰ نمونه، ۴ مرکز، random_state=7, cluster_std=1.8)

دیتا بسازید. دو نمودار کنار هم رسم کنید: یکی خاکستری و بدون برچسب (چیزی که الگوریتم می‌بیند)، یکی رنگی بر اساس برچسب واقعی.

9.2_ دو ستون عددی bmi و blood_glucose_level رو از diabetes_prediction_dataset1.csv انتخاب کنید و یک نمودار پراکندگی خاکستری و بدون برچسب از شون رسم کنید (دقیقاً مثل چیزی که یک الگوریتم خوشه‌بندی می‌بیند).

9.3_ با نگاه به نموداری که در سؤال 9.2 رسم کردید، در یکی دو جمله بگید آیا خوشه‌های طبیعی و مجزا با چشم قابل تشخیص یا نه، و این چه فرقی با نمودار تمیز و مرتب make_blobs (سؤال 9.1) داره.